In Python, object-oriented Programming (OOPs) is a programming paradigm(set) that uses **objects and classes** in programming. It aims to implement real-world entities like

inheritance,

polymorphisms,

encapsulation, Abstraction etc. in the programming.

The main concept of OOPs is to **bind** the data and the functions that work together as a single unit.

OOPS:

Structure of a Class

```python
class Vehicle:
    def get_info(self, make, model, year, county):
        return year, make, model, county

    def drive(self, make, model, year, county):
        return make + " " + model + " can drive"

car = Vehicle()

print(car.get_info("Tata", "Punch", "2020", "India"))
print(car.drive("Tata", "Punch", "2020", "India"))
```

Classes: Classes are contains

- **methods (functions)**
- **attributes (data, Variables)**
- **Logical Structure or all**
- **blueprints** for creating objects.

Object:

- Objects are instances of classes.
- created using the class as a template.
- Using object we can access the
  A) Attribute(data)
  b) Methods(Function) or both of a Class

Inheritance

The derived class(Child Class) inherits the properties of the Base class(Parent Class)
In the Inheritance Derived class(Child Class) can access all the properties (Attributes and Functions) of the Base class(Parent Class).

There are different types of Inheritance
1) **Single Inheritance**
2) **Multilevel Inheritance**
3) **Multiple Inheritance**

1) Single Inheritance







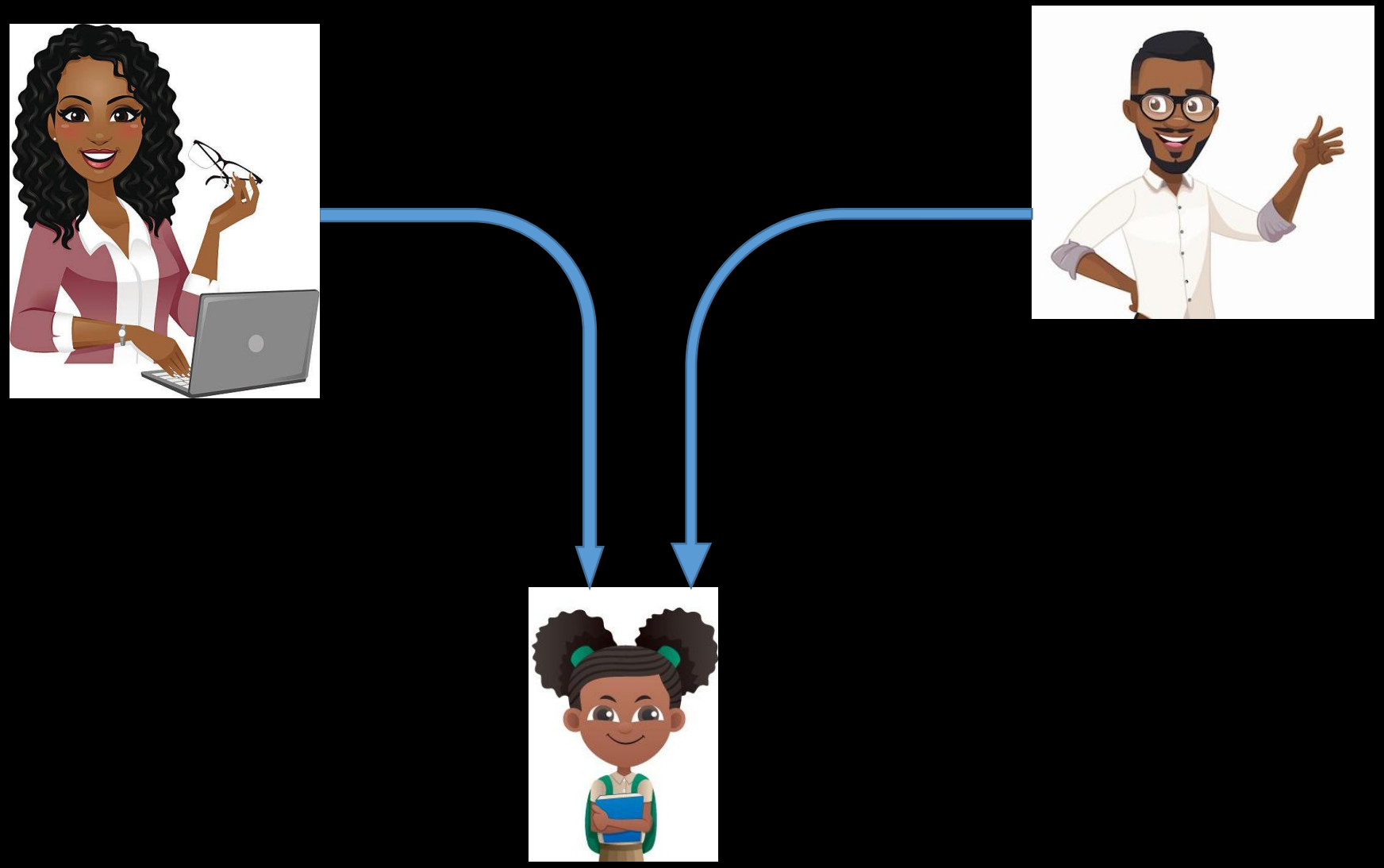
2) Multilevel Inheritance



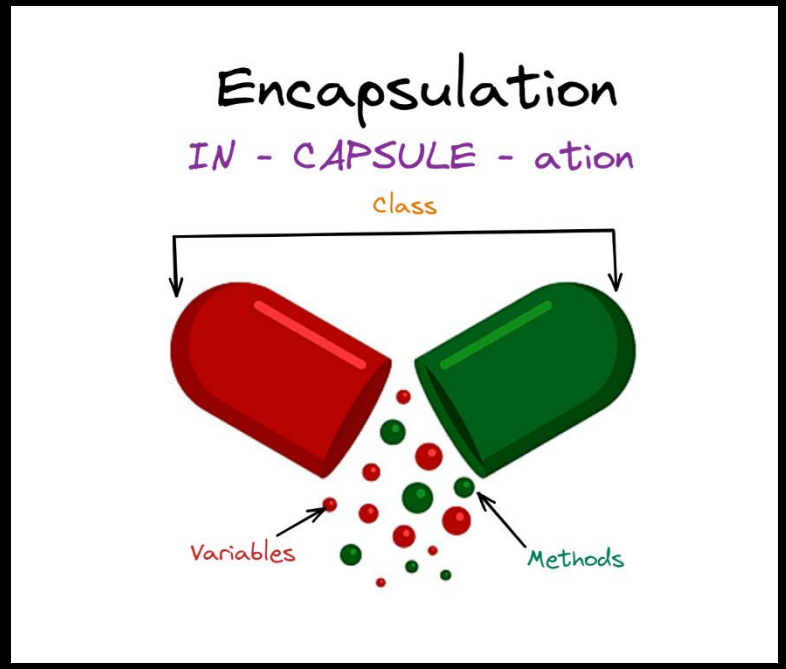




3) Multiple Inheritance



Encapsulation:



Poly morphism:

Polymorphism means creating multiple forms, either by passing multiple argument in a method or use of same method but with different datatype, arguments or operation.

There are two polymorphisms.

1) Method overloading,
2) Method overriding,

Method Overloading

If a method is written in such a way that it can perform more than one task, it is called method overloading. We can achieve method overloading by writing the same method with several parameters.

```
class myClass:
    def sum(self, a=None,b=None,c=None):
        if a !=None and b !=None and c !=None:
            s = a+b+c
        elif a !=None and b !=None:
            s = a+b
        else:
            s = "provide altleast two numbers"
        return s

obj = myClass()
obj.sum(10,5)
```

```
15
```

Method Overriding:

In inheritance, if we write **the same method** in both classes, I.e. parent class as well as child class then the parent class method is not available in the child class. In this case child class method replaces the parent class Method. Method overriding is used when the programmer wants to modify the existing behaviour of a method.

```
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

class Cat(Animal):
    def sound(self):
        print("Cat meows")

# Create instances of Dog and Cat
dog = Dog()
cat = Cat()

# Call the sound method for each instance
dog.sound()  # Output: Dog barks
cat.sound()  # Output: Cat meows
```

Method Overriding with super keyword:

However, if a programmer still wants to use the Parent class method then, the same can be achieved by using the **"super" keyword.** As shown in example two.

```
class Animal:
    def speak(self):
        print("The animal makes a sound")

class Dog(Animal):
    def speak(self):
        super().speak()  # Call the speak method from the superclass
        print("The dog barks")

# Create an instance of Dog and call the speak method
dog = Dog()
dog.speak()
```

```
The animal makes a sound
The dog barks
```

```python
class Vehicle:
    def get_info(self, make, model, year, county):
        return year, make, model, county

    def drive(self, make, model, year, county):
        return make + " " + model + " can drive"

car = Vehicle()

print(car.get_info("Tata", "Punch", "2020", "India"))
print(car.drive("Tata", "Punch", "2020", "India"))
```

```python
class Vehicle:
    def __init__(self, make, model, year, county):
        self.make = make
        self.model = model
        self.year = year
        self.county = county

    def get_info(self):
        return self.year, self.make, self.model, self.county

    def drive(self):
        return self.make + " " + self.model + " can drive"

car = Vehicle("Tata", "Punch", "2020", "India")

print(car.get_info())  # Output: ('2020', 'Tata', 'Punch', 'India')

print(car.drive())  # Output: Tata Punch can drive
```

END OF OOPS